

Documentation Technique

Application *DPE Vision IA*

1 Objectif

L'application **DPE Vision IA** permet d'analyser et de prédire la **performance énergétique (étiquette DPE)** et la **consommation annuelle** des logements du département du Rhône à partir des données publiques **ADEME** et **Enedis**.

Elle repose sur trois modules principaux :

1. Une **API FastAPI** pour la prédiction et l'interprétation IA.
2. Une **interface Streamlit** pour la visualisation et l'interaction utilisateur.
3. Une **infrastructure Docker** pour le déploiement reproductible et simplifié.

2 Architecture du système

2.1 Schéma d'architecture global

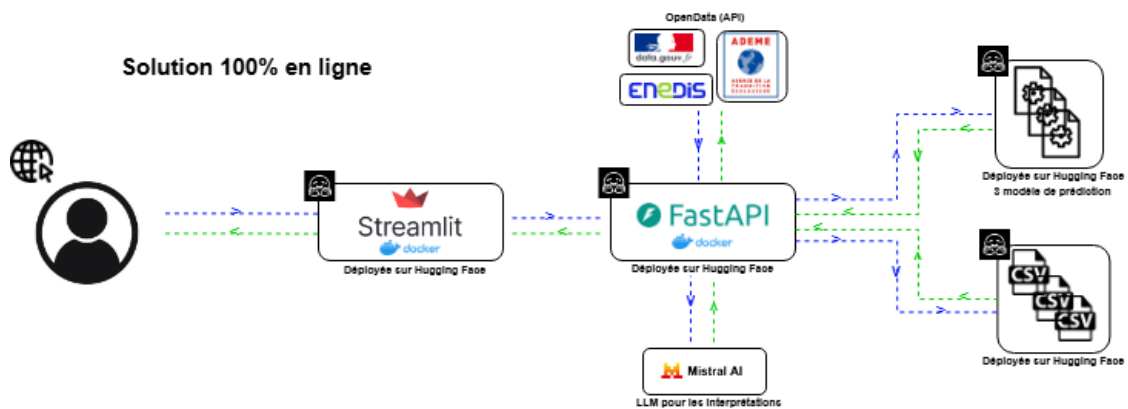


FIGURE 1 – Schéma global de l'application DPE Vision IA (Rhône 69)

Ce schéma illustre l'organisation générale des composants du projet :

- L'utilisateur interagit avec l'application **Streamlit** hébergée sur Hugging Face.
- Streamlit envoie des requêtes à l'**API FastAPI** (autre service Hugging Face).
- L'API interroge les **modèles de Machine Learning** (compressés) pour prédire la classe DPE et la consommation.
- Les résultats sont interprétés par **Mistral AI** pour générer une explication textuelle.
- Les données ADeme et Enedis sont traitées, nettoyées et fusionnées localement avant publication.

3 Chaîne de traitement des données

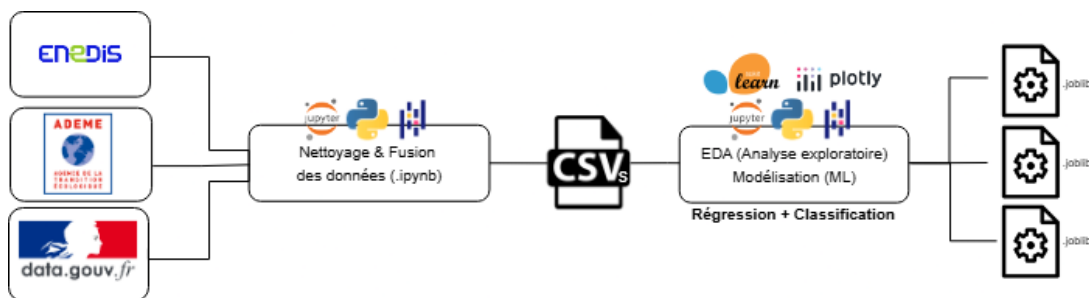


FIGURE 2 – Pipeline de traitement des données ADEME & Enedis

Les principales étapes de la chaîne de traitement sont :

1. **Collecte** des données depuis les APIs **ADEME**, **Enedis** et **Data Gouv**.
2. **Nettoyage et fusion** : suppression des doublons, homogénéisation des communes, conversions d'unités.
3. **Analyse exploratoire (EDA)** et visualisation des distributions via Plotly.
4. **Modélisation Machine Learning** avec scikit-learn :
 - Régression : prédiction de la consommation énergétique.
 - Classification : prédiction du DPE sans puis avec consommation.
5. Export du dataset final (CSV/Parquet) utilisé dans FastAPI et Streamlit.

4 Installation sur poste local (Docker)

4.1 Prérequis

- Docker Desktop (version ≥ 24.0)
- Connexion internet active
- (Optionnel) Python ≥ 3.10 si exécution sans Docker

4.2 Clonage du dépôt

```
git clone https://github.com/riadshrn/m2_enedis_dpe_app.git
cd m2_enedis_dpe_app
```

4.3 Lancement avec Docker Compose

```
docker compose up --build
```

Premier lancement : environ 1 à 2 minutes pour la construction des images.

4.4 Accès aux services

Service	URL locale	Description
Streamlit App	http://localhost:8501	Interface utilisateur
FastAPI	http://localhost:8000	API de prédiction
Swagger UI	http://localhost:8000/docs	Documentation API

5 Environnement logiciel et bibliothèques

5.1 Backend – API (FastAPI)

Package	Version	Description
fastapi	0.115.4	Framework web Python asynchrone
uvicorn	0.32.0	Serveur ASGI pour FastAPI
pandas	2.2.3	Manipulation de données
scikit-learn	1.5.2	Modélisation et apprentissage automatique
joblib	1.4.2	Sérialisation des modèles
requests	2.32.3	Requêtes HTTP (API / Mistral)
tqdm	4.66.5	Barre de progression
pyarrow	17.0.0	Format de données Parquet
fastparquet	2024.5.0	Lecture/écriture de fichiers Parquet

5.2 Frontend – Interface Streamlit

Package	Version	Description
streamlit	1.38.0	Interface web interactive
pandas	2.2.3	Manipulation de DataFrame
plotly	5.24.1	Visualisation interactive
requests	2.32.3	Communication avec FastAPI
kaleido	0.2.1	Export d'images Plotly (PNG)
streamlit_plotly_events	0.0.6	Interaction entre graphiques et Streamlit

6 Déploiement Hugging Face

Composant	URL
API FastAPI	https://riadshrn-api-dpe-conso.hf.space/docs
App Streamlit	https://riadshrn-streamlit-dpe-app.hf.space

7 Conclusion

L'application **DPE Vision IA** constitue une solution complète pour la prédiction et l'analyse énergétique territoriale, combinant les forces de FastAPI, Streamlit et du Machine Learning. Le déploiement sur Hugging Face permet une **reproductibilité**, une **accessibilité publique** et une **interopérabilité API-Frontend** exemplaires.